

Optimization (SGD)

Gunhee Kim



서울대학교
SEOUL NATIONAL UNIVERSITY

Outline

- Basic Algorithms
- Algorithms with Adaptive Learning Rates
- Parameter Initialization

Vector Derivatives

- Regular derivative

$$\frac{dy}{dx} \in \mathbb{R}$$

- Scalar to scalar: $x, y \in \mathbb{R}$
- If x changes by a small amount, how much will y change?

- Gradient

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_N} \right]^T \in \mathbb{R}^N$$

- Vector to scalar: $\mathbf{x} \in \mathbb{R}^N, y \in \mathbb{R}$
- For each dim of $\mathbf{x}$, if it changes by a small amount then how much y change?

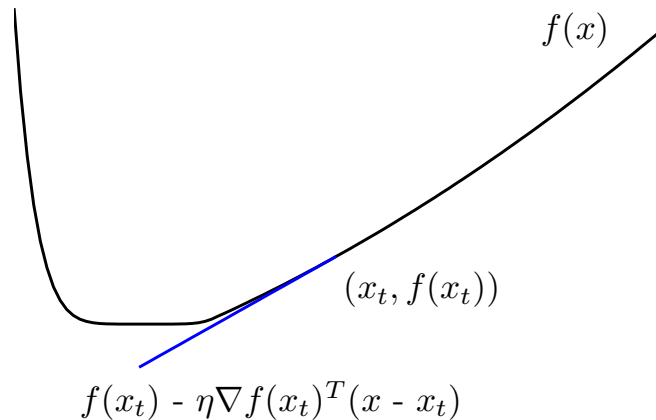
- Jacobian

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \left[\frac{\partial y_m}{\partial x_n} \right]_{n,m} \in \mathbb{R}^{N \times M}$$

- Vector to Vector: $\mathbf{x} \in \mathbb{R}^N, \mathbf{y} \in \mathbb{R}^M$
- For each dim of $\mathbf{x}$, if it changes by a small amount then how much each dim of $\mathbf{y}$ change?

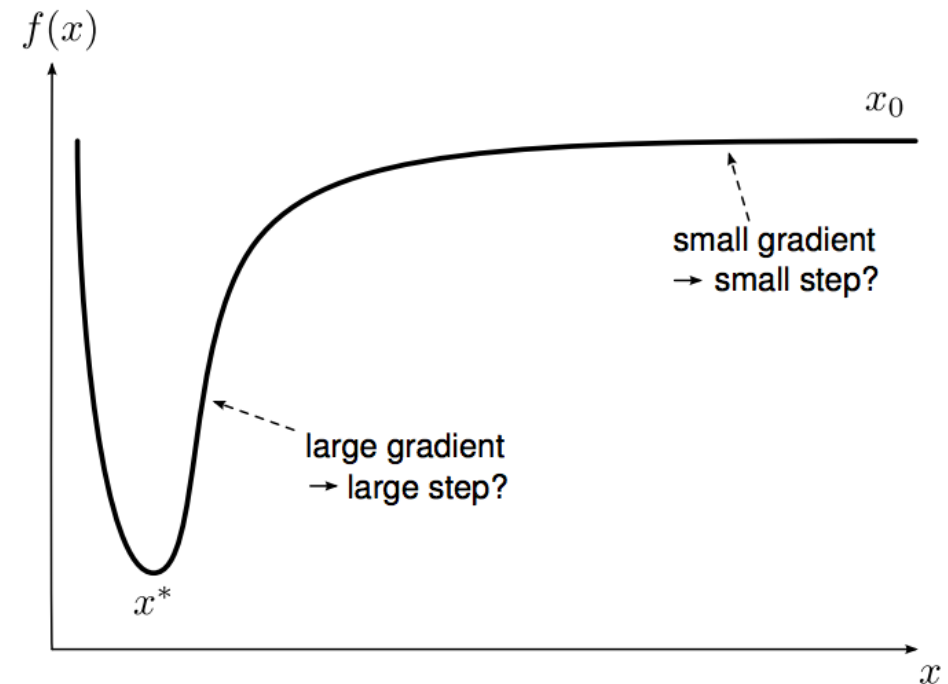
Gradient Descent

- The (almost) simplest algorithm in the world
 - Although it may not be often the most efficient method
- Gradient $\partial f(x)/\partial x$ at x is the direction where $f(x)$ increases
 - The negative $-\partial f(x)/\partial x$ is called steepest descent direction



Gradient Descent

- Goal: minimize $f(\mathbf{w})$
- Procedure
 - Start from initial point $\mathbf{w}_0$
 - Just iterate $\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \nabla J(\mathbf{w}_k)$
 - α_k is a learning rate (stepsize) at iteration k
- The learning rate α
 - The most important hyperparameter in practice



Batch Gradient Descent

- Find a parameter set θ to minimize error function $J(\theta)$

$$\theta_{k+1} = \theta_k - \varepsilon_k \nabla J(\theta_k)$$

- Batch (deterministic) gradient descent
 - Process all examples together in each step

$$\theta_{k+1} = \theta_k - \varepsilon_k \mathbf{g} \text{ where } \mathbf{g} = \frac{1}{m} \nabla_{\theta} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

- Entire training set examined at each step
- Very slow when m is very large

Stochastic Gradient Descent

- Computing the exact gradient is expensive
 - This seems wasteful because there will be only a small change in the weights
- Batch (deterministic) GD $\rightarrow$ SGD
 - Each batch contains just one example
 - Much faster than exact gradient descent
 - Effective when combined with momentum
- Select examples randomly (or reorder and choose in order)
 - for $i = 1$ to m :

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \varepsilon_k \boldsymbol{g} \text{ where } \boldsymbol{g} = \nabla_{\boldsymbol{\theta}} L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$$

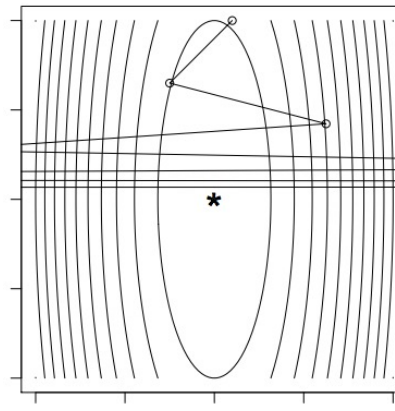
Stochastic Gradient Descent

- Does it converge?
 - When the learning rate decreases with an appropriate rate and (with mild assumptions), SGD converges

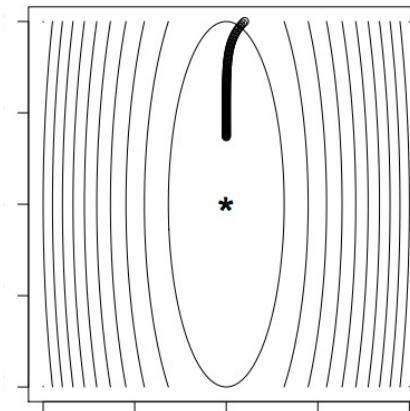
$$\sum_{k=1}^{\infty} \varepsilon_k = \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \varepsilon_k^2 < \infty$$

- The learning rate (or step size) is a free parameter
 - No general prescriptions for selecting appropriate learning rate
 - Even no fixed rate appropriate for entire learning period

Too large size:
Divergence



Too small size:
Slow convergence

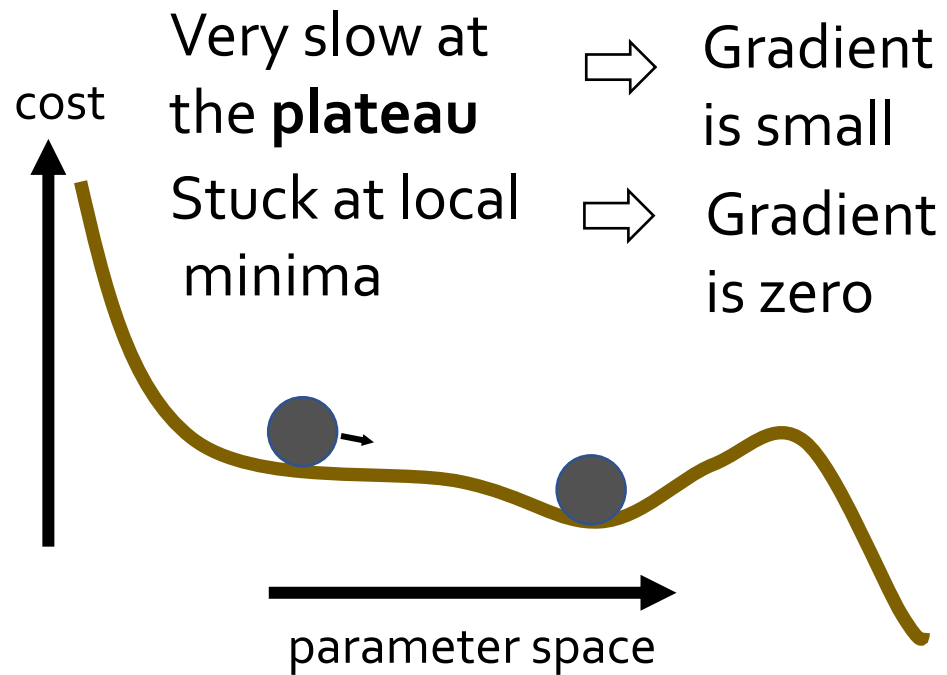


Mini-Batch Gradient Descent

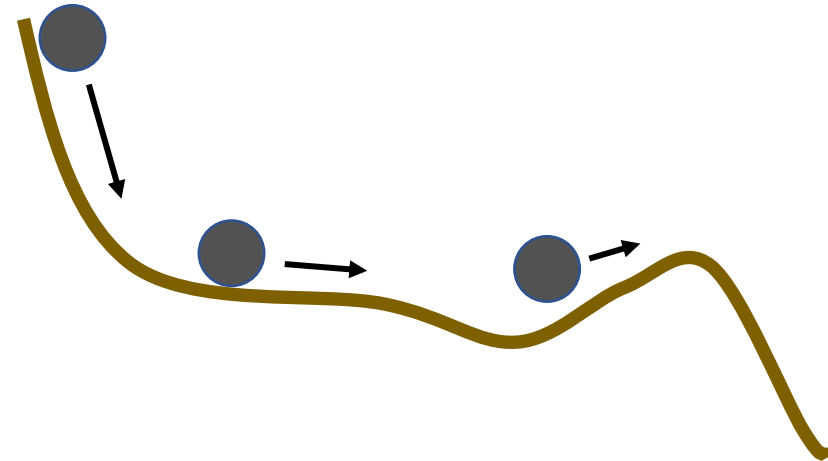
- Mini-batch optimization
 - Divide the dataset into small batches of examples, compute the gradient using a single batch, make an update, then move to the next batch
 - Good for multicore or parallel architectures
 - Particularly good for GPU that is very good at matrix computation (power of 2 batch sizes)
 - Small batches can offer a regularizing effect (due to noise by random sampling)
- Convergence is very sensitive to learning rate
 - Oscillations near solution due to probabilistic nature of sampling
 - Need to decrease with time to ensure the algorithm converges

Gradient Descent with Momentum

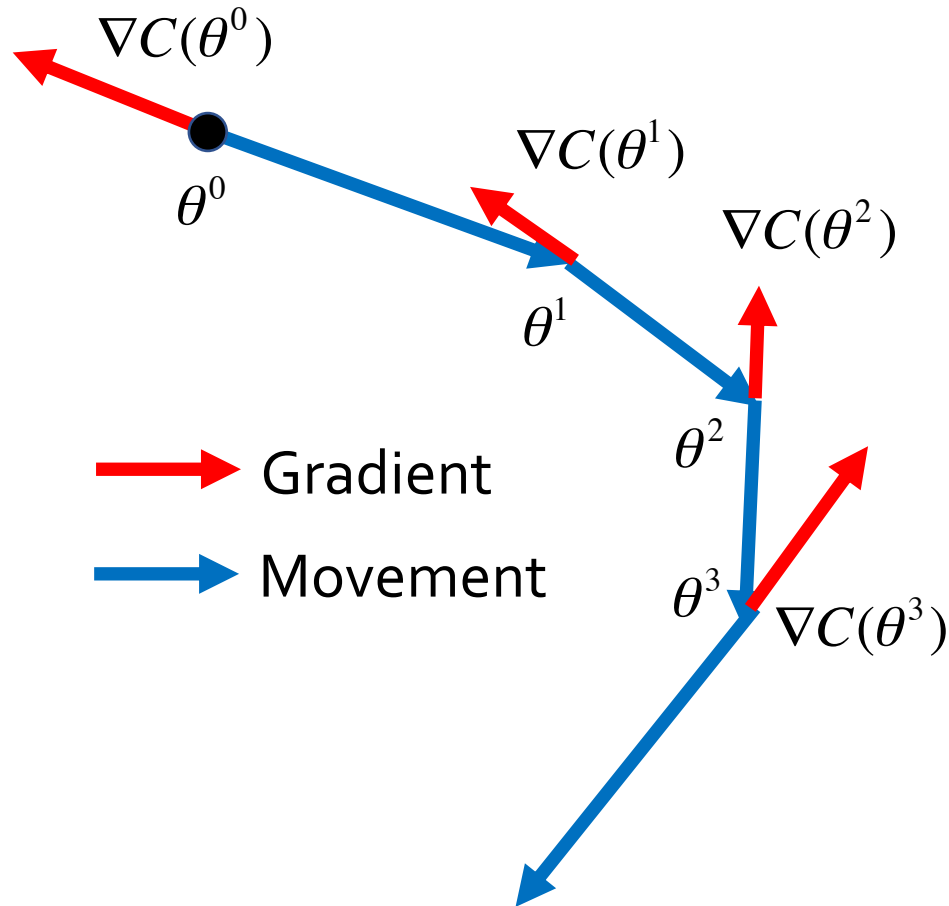
Without momentum



With momentum



Original Gradient Descent



Start at position θ^0

Compute gradient at θ^0

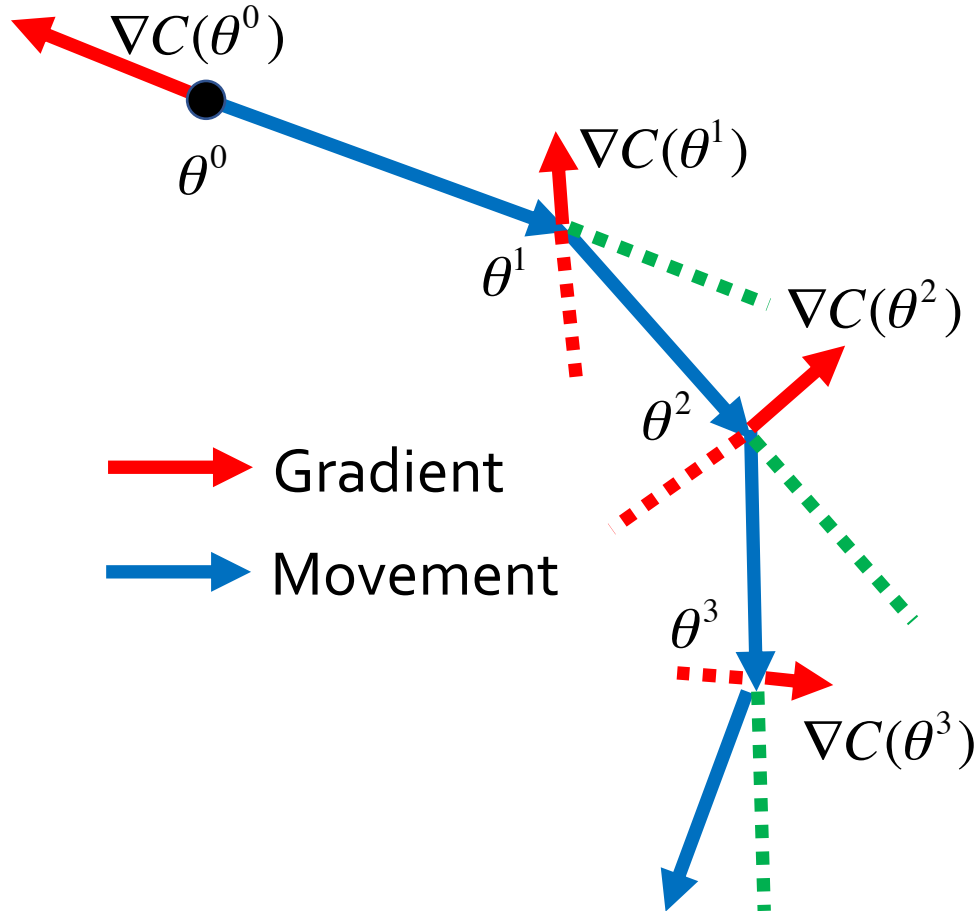
Move to $\theta^1 = \theta^0 - \varepsilon \nabla C(\theta^0)$

Compute gradient at θ^1

Move to $\theta^2 = \theta^1 - \varepsilon \nabla C(\theta^1)$

$\vdots$

Gradient Descent with Momentum



Start at position θ^0

Momentum $v^0 = 0$

Compute gradient at θ^0

Momentum $v^1 = \lambda v^0 - \varepsilon \nabla C(\theta^0)$

Move to $\theta^1 = \theta^0 + v^1$

Compute gradient at θ^1

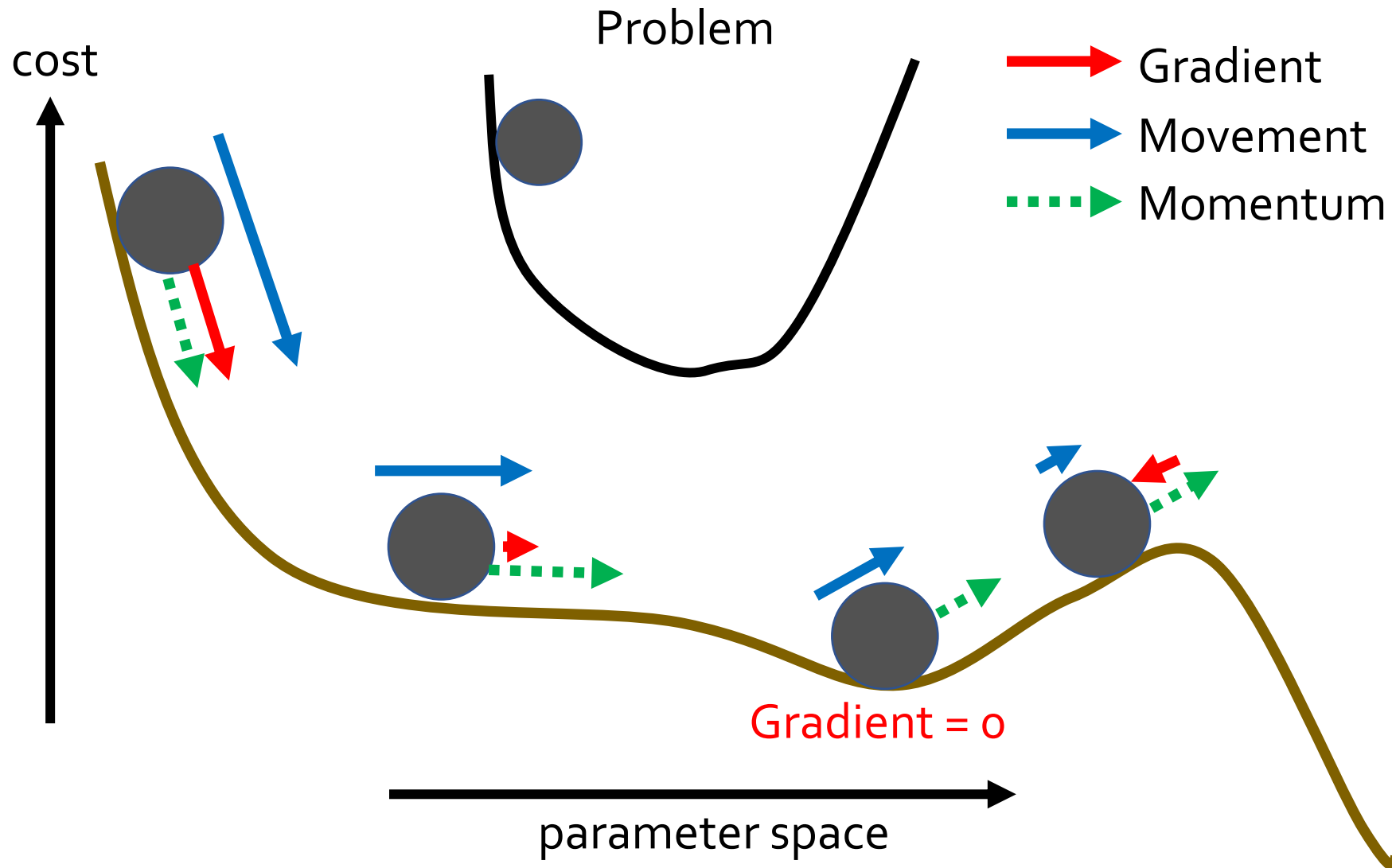
Momentum $v^2 = \lambda v^1 - \varepsilon \nabla C(\theta^1)$

Move to $\theta^2 = \theta^1 + v^2$

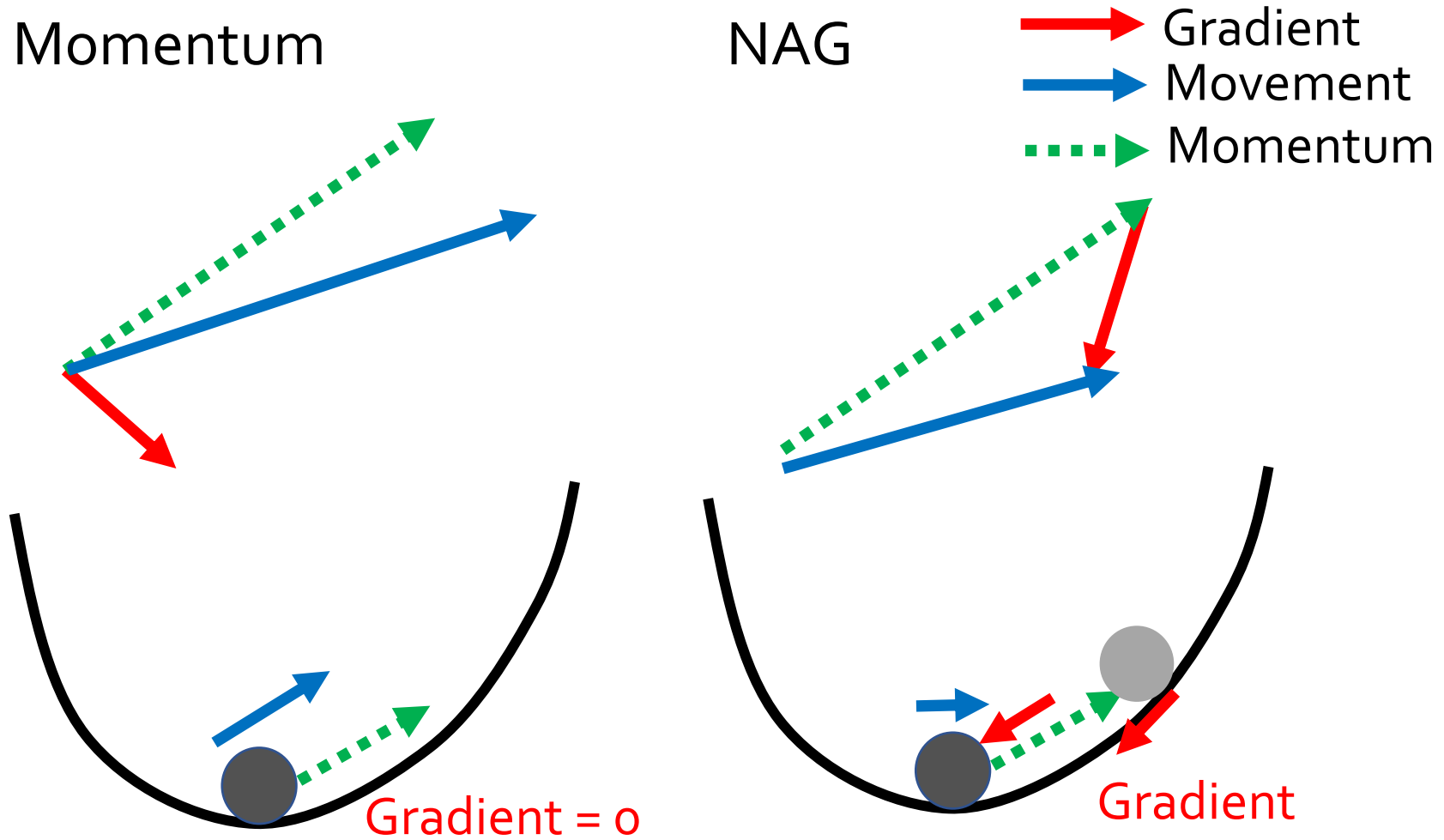
$\vdots$

- v^i is the weighted sum of all the previous gradient $(\nabla C(\theta^0), \nabla C(\theta^1), \dots, \nabla C(\theta^{i-1}))$

Gradient Descent with Momentum



Nesterov's Accelerated Gradient



- Do not compute the gradient at old state

Gradient descent, Momentum, NAG

- SGD

- Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
- Apply update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \varepsilon \mathbf{g}$

- SGD with momentum

- Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
- Compute the velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \mathbf{g}$
- Apply update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$

A minibatch of m
training examples:
 $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}$

- SGD with Nesterov momentum

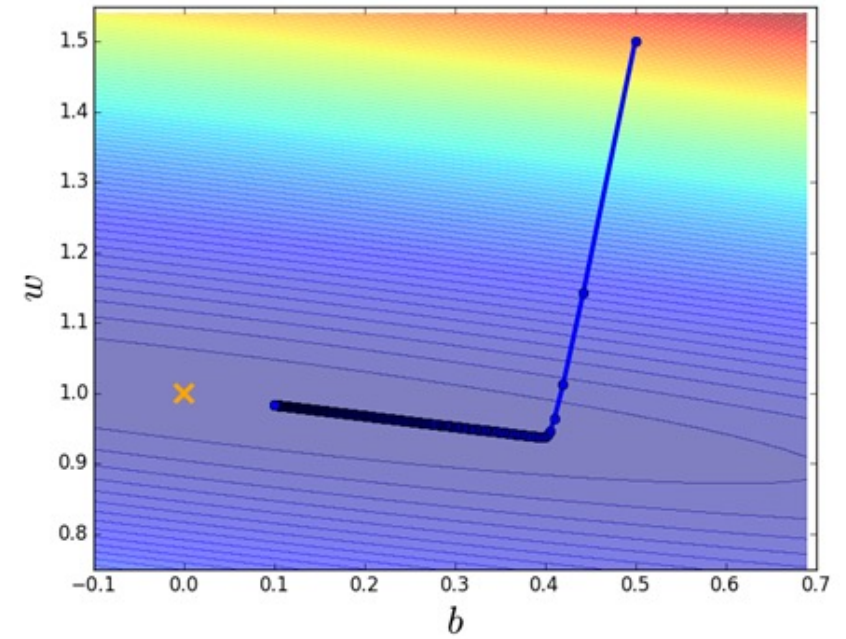
- Apply interim update: $\hat{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \mathbf{v}$
- Compute gradient at interim point: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\hat{\boldsymbol{\theta}}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \hat{\boldsymbol{\theta}}), y^{(i)})$
- Compute the velocity and update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \mathbf{g}$ and $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$

Outline

- Basic Algorithms
- Algorithms with Adaptive Learning Rates
- Parameter Initialization

How to Set Learning Rates

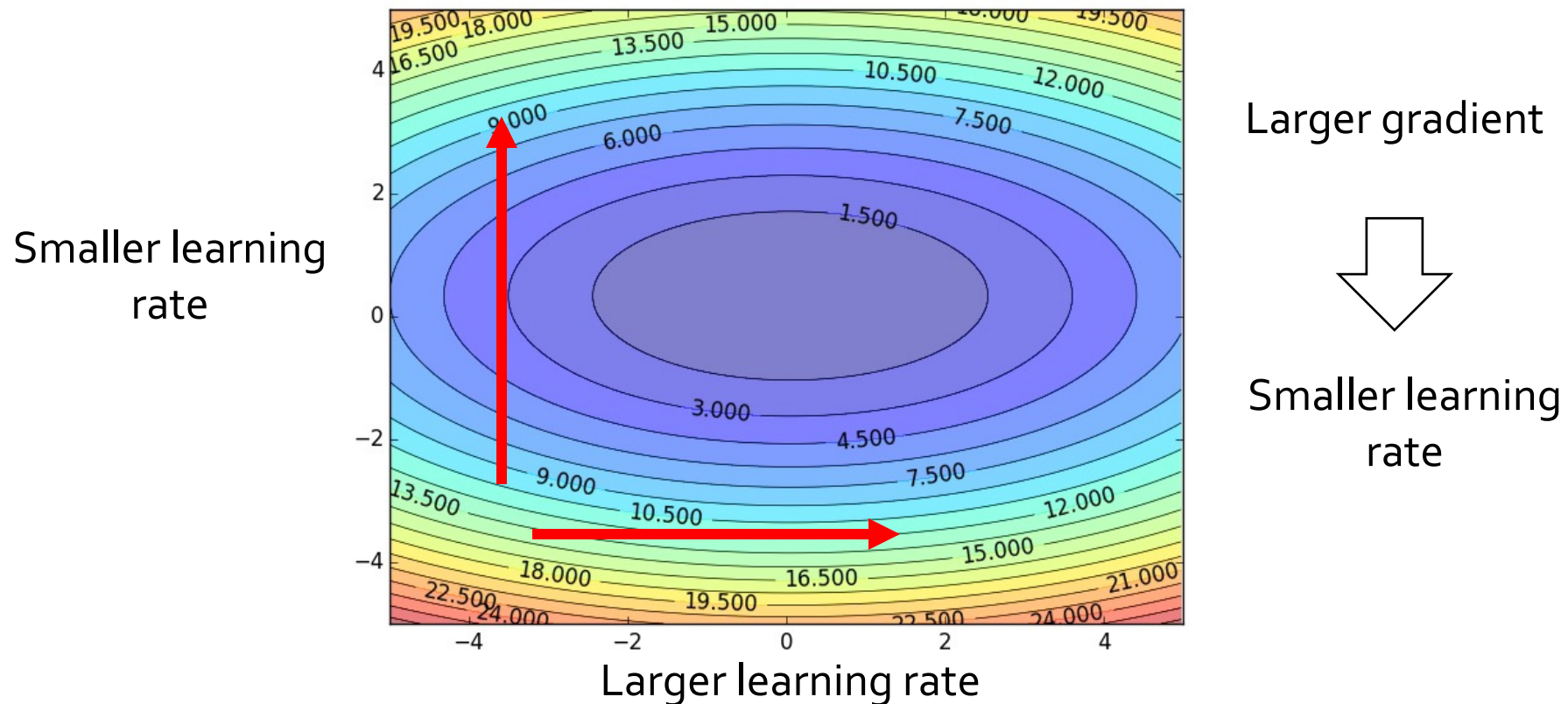
- One of the most difficult hyperparameters to set
- Assumption: reduce the learning rate by some factor every few epochs
 - At the beginning, we are far from a minimum, so we use a larger learning rate
 - After several epochs, we are close to a minimum, so we reduce the learning rate
 - $1/t$ decay: $\varepsilon = \varepsilon_0 / (1 + kt)$ where t is the iteration number, and ε_0, k are hyperparameters
 - Exponential decay: $\varepsilon = \varepsilon_0 \exp(-kt)$



Not always true

Adaptive Learning Rates

- Each parameter should have different learning
 - Automatically adapt the axis-aligned learning rates throughout the course of learning



Adagrad

- Different adaptive learning rates for each weight
 - Divide the learning rate element-wise by history of *average* gradient
 - If w has small average gradient $\rightarrow$ large learning rate
 - If w has large average gradient $\rightarrow$ small learning rate
- Loop
 - Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
 - Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$
 - Apply update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \varepsilon / (\delta + \sqrt{\mathbf{r}}) \odot \mathbf{g}$
- Empirical behavior
 - The accumulation of squared gradients from the beginning can cause an excessive decrease in the learning rate

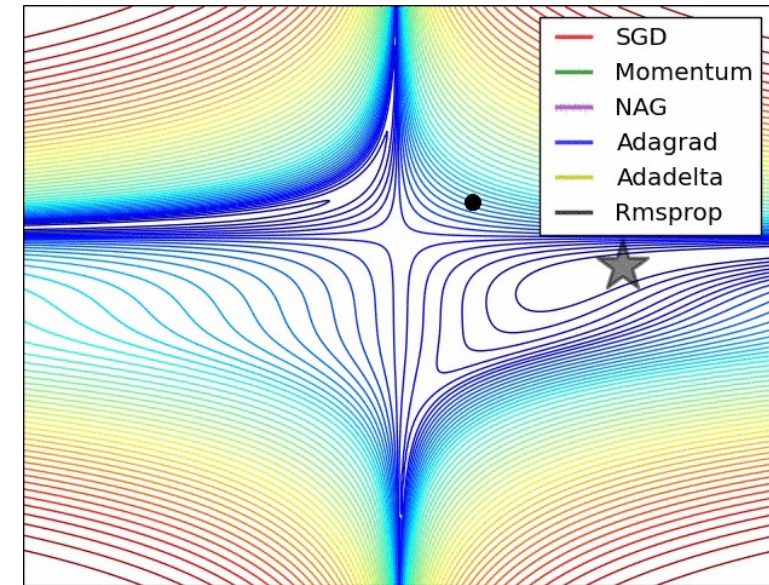
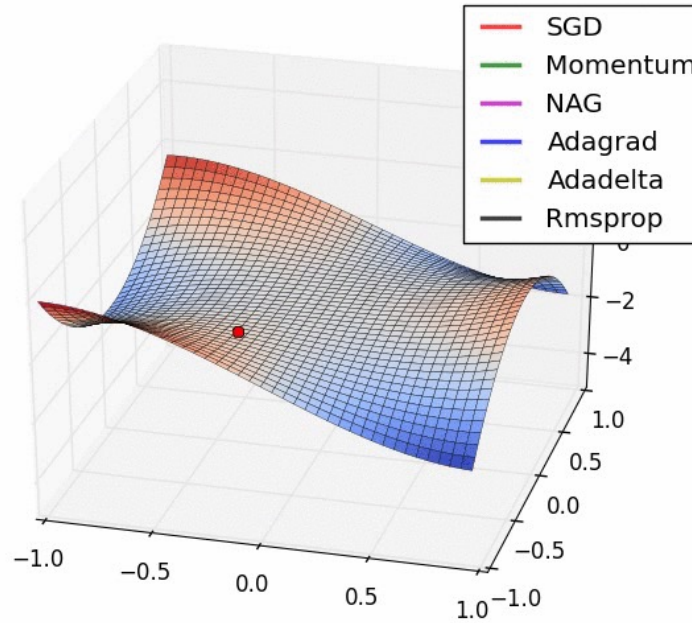
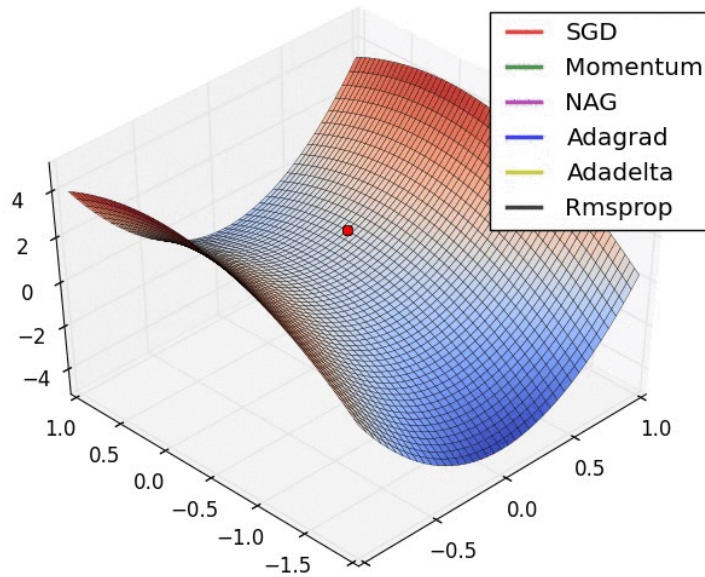
RMSprop

- One of the go-to optimization method for deep learning
 - Suggested by G. Hinton in the Coursera course lecture 6
 - Problem of AdaGrad: shrink the learning rate according to the entire history of the squared gradient (too small before arriving)
 - Exponentially decaying average to discard history from the extreme past
 - Still modulates the learning rate of each weight
- Loop
 - Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
 - Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$
 - Apply update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \varepsilon / (\sqrt{\delta + \mathbf{r}}) \odot \mathbf{g}$

Adam (Adaptive Moment)

- RMSProp + momentum
 - Consider both first-order and second-order moments
 - Include bias correction
- Loop
 - Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$
 - Update the first/second moment: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$ and $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$ where ρ_1/ρ_2 : exponential decay rate
 - Correct biases: $\hat{\mathbf{s}} \leftarrow \mathbf{s} / (1 - \rho_1^t)$ and $\hat{\mathbf{r}} \leftarrow \mathbf{r} / (1 - \rho_2^t)$
 - Apply update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \varepsilon \hat{\mathbf{s}} / (\sqrt{\hat{\mathbf{r}}} + \delta) \odot \mathbf{g}$

Visualizing Optimization Algorithms



Outline

- Basic Algorithms
- Algorithms with Adaptive Learning Rates
- **Parameter Initialization**

Parameter Initialization

- Only heuristic recommendation
 - Neural network optimization is not yet well understood
 - How do we set the initial point?
 - How does the initial point affect generalization?
- Heuristics #1: *Break symmetry* between different units
 - The units at the same layers should be initialized differently
 - Otherwise, they are constantly updated in the same way
 - Random initialization (much cheaper and good enough in a high-entropy distribution in a high-D space)

Parameter Initialization

- Heuristics #2: Simply drawn from a Gaussian or uniform
 - However, magnitudes and scales matter
- Trade-off for larger initial weights
 - Help avoid losing signal during forward/back-propagation
 - May cause exploding values, sensitivity to small perturbation, and loss of gradient through saturated units
 - Smaller values encourage regularization

Parameter Initialization

- Other parameter settings are easier
 - Simply set the biases to zero
 - Safely initialize variance or precision parameters to 1
- Practical tips (from pre-training and fine-tuning)
 - Initialize a supervised model with the parameters learned by an unsupervised model trained on the same inputs (e.g., autoencoders, greedy layer-wise training)
 - Use the parameters learned on a related task
 - Sometimes, the parameters on an unrelated task may help
 - Other tips: regard multiple settings as hyper-parameters, and test with a single mini-batch of data